

Projet MOGPL 2025 : La balade du robot

Rapport de projet

Yanis Labiadh
David Gu

Table des matières

1	Introduction	2
2	Question 1(a) : Modélisation en graphe orienté	2
2.1	Transitions	2
3	Question 1(b) : Algorithme de résolution et complexité	2
3.1	Pourquoi BFS?	2
3.2	Complexité	2
4	Question 1(c) : Expériences en fonction de la taille de la grille	3
5	Question 1(d) : Expériences en fonction du nombre d'obstacles	3
6	Question 1(e) : Génération automatique d'instances avec un PLNE	3
6.1	Contraintes	4
6.2	Fonction objectif	4
6.3	Interface utilisateur	4

1 Introduction

Ce projet étudie le déplacement optimal d'un robot dans un dépôt représenté sous forme de grille comportant des obstacles. Le robot se déplace entre les intersections valables de la grille, possède une orientation, et dispose de commandes élémentaires permettant de tourner ou d'avancer d'une, deux ou trois unités.

L'objectif est de déterminer la séquence optimale minimisant le temps total, chaque commande durant exactement une seconde. Le projet comporte cinq volets :

- modélisation du problème sous forme de graphe orienté ;
- conception d'un algorithme de résolution efficace ;
- expériences sur la complexité en fonction de la taille de la grille ;
- expériences sur l'influence du nombre d'obstacles ;
- génération automatique d'instances via un programme linéaire (PLNE).

2 Question 1(a) : Modélisation en graphe orienté

Le déplacement du robot est modélisé par un graphe orienté dont chaque nœud correspond à un état :

$$(i, j, d)$$

où (i, j) est une intersection valide de la grille (c'est-à-dire un point où les quatre cases environnantes sont libres), et $d \in \{N, E, S, O\}$ représente l'orientation.

2.1 Transitions

Depuis un état (i, j, d) , cinq actions sont possibles :

- **Tourner à gauche** : $(i, j, d) \rightarrow (i, j, d_g)$
- **Tourner à droite** : $(i, j, d) \rightarrow (i, j, d_d)$
- **Avancer de 1 case** : $(i, j, d) \rightarrow (i_1, j_1, d)$
- **Avancer de 2 cases** : $(i, j, d) \rightarrow (i_2, j_2, d)$
- **Avancer de 3 cases** : $(i, j, d) \rightarrow (i_3, j_3, d)$

Les déplacements en avant ne sont autorisés que si toutes les intersections intermédiaires sont valides.

L'état initial est (D_1, D_2, d_0) et l'état final peut être atteint avec n'importe quelle orientation en (F_1, F_2) .

3 Question 1(b) : Algorithme de résolution et complexité

Comme chaque commande a exactement un coût unitaire, le problème revient à trouver un plus court chemin en nombre d'étapes. Le **parcours en largeur (BFS)** est l'algorithme le plus adapté dans ce contexte.

3.1 Pourquoi BFS ?

- Tous les arcs du graphe ont le même coût (1 seconde).
- BFS garantit le plus court chemin en nombre d'arcs.
- Le graphe n'est pas construit en entier : les successeurs sont générés dynamiquement, ce qui économise la mémoire.
- La structure du graphe étant implicite, BFS l'exploite naturellement sans surcharge.

3.2 Complexité

L'espace des états comporte au plus :

$$|V| = 4MN.$$

Le nombre d'arcs est proportionnel à $|V|$ (au plus cinq transitions par état).

Ainsi, la complexité en temps du BFS est :

$$O(|V| + |E|) = O(MN).$$

La mémoire utilisée est également linéaire :

$$O(MN).$$

4 Question 1(c) : Expériences en fonction de la taille de la grille

Nous faisons varier la taille d'une grille $N \times N$ pour étudier l'impact sur le temps d'exécution du BFS. Les temps affichés correspondent à la moyenne sur 10 instances générées aléatoirement.

Taille N	Temps moyen (s)
10	0.000870
20	0.000683
30	0.000187
40	0.000431
50	0.000174

TABLE 1 – Temps moyen du BFS en fonction de la taille de la grille.

Analyse. Les temps d'exécution restent extrêmement faibles. Le comportement attendu en $O(N^2)$ est masqué par des phénomènes aléatoires comme la proximité de l'objectif ou la présence de couloirs bloqués. Le BFS est si efficace que même des tailles moyennes restent instantanées.

5 Question 1(d) : Expériences en fonction du nombre d'obstacles

Pour une grille fixe 20×20 , nous faisons varier le nombre d'obstacles P .

Obstacles P	Temps moyen (s)
10	0.000215
20	0.000795
30	0.002307
40	0.004543
50	0.007064

TABLE 2 – Temps moyen du BFS pour une grille 20×20 en fonction du nombre d'obstacles.

Analyse. Le temps croît clairement avec P . Cela s'explique par :

- un nombre plus important d'états explorés avant de trouver un passage ;
- davantage de détours dans les zones encombrées ;
- des situations plus fréquentes où aucun chemin n'existe.

6 Question 1(e) : Génération automatique d'instances avec un PLNE

Nous utilisons Gurobi pour générer automatiquement des dépôts comportant exactement P obstacles, placés selon des contraintes géométriques et structurelles.

Variables

$$x_{i,j} = \begin{cases} 1 & \text{si la case } (i,j) \text{ contient un obstacle,} \\ 0 & \text{sinon.} \end{cases}$$

6.1 Contraintes

— Nombre total d'obstacles :

$$\sum_{i,j} x_{i,j} = P.$$

— Distribution équilibrée :

$$\sum_j x_{i,j} \leq \frac{2P}{M}, \quad \sum_i x_{i,j} \leq \frac{2P}{N}.$$

— Interdiction des motifs 1-0-1 :

$$x_{i,j} + x_{i,j+2} \leq 1 + x_{i,j+1}.$$

On note que pour les contraintes de distribution équilibrée, il existe des cas tel que le modèle est rendu infaisable : quand N ou M sont trop grands pour un P trop petit on obtient des valeurs < 1 , il ne peut pas il y avoir d'obstacles de par cette contrainte cependant il faut P obstacles.

On propose alors de modifier la contrainte de la manière suivante :

$$\sum_j x_{i,j} \leq \lceil \frac{2P}{M} \rceil, \quad \sum_i x_{i,j} \leq \lceil \frac{2P}{N} \rceil.$$

Ainsi, il sera possible pour toute taille de N, M, P d'avoir une solution.

6.2 Fonction objectif

Un poids aléatoire $w_{i,j}$ est associé à chaque case. On minimise :

$$\min \sum_{i,j} w_{i,j} x_{i,j}.$$

6.3 Interface utilisateur

Une interface complète a été implémentée :

1. choix de la taille (N, M) ;
2. choix du nombre d'obstacles P ;
3. génération automatique de la grille via Gurobi ;
4. saisie de la position de départ, orientation initiale et objectif ;
5. affichage de la grille générée ;
6. exécution immédiate du BFS et affichage :
 - du nombre minimal de commandes ;
 - de la séquence optimale ;
 - de la validité ou non de l'atteinte du but.

Cette interface permet de tester de nombreux scénarios sans effort et d'explorer des grilles très différentes, optimisées par le PLNE.

A cela s'ajoute une deuxième interface permettant quant à elle d'exécuter notre algorithme BFS sur une instance contenue dans un fichier.